Architecting the Memory-Compute Interface: Realizing Neural Network Execution within Random-Access Memory

1. Introduction to the Memory-Centric Computing Paradigm

The prevailing von Neumann computing architecture, characterized by the strict physical and logical separation of the central processing unit (CPU) and the memory hierarchy, has governed digital system design for decades. However, the exponential scaling of data-intensive workloads, specifically deep neural networks (DNNs) and large language models (LLMs), has exposed a fundamental architectural flaw commonly referred to as the memory wall[1]. In these computational workloads, the energy and latency costs associated with shuttling data—weights, activations, and partial sums—back and forth across the memory bus vastly overshadow the energy cost of the arithmetic operations themselves[4]. Modern host processors, including graphics processing units (GPUs) and tensor processing units (TPUs), possess immense computational density but are frequently starved by the limited bandwidth of the memory channel, resulting in severe underutilization of silicon and prohibitive power consumption[6].

To fundamentally circumvent this data movement bottleneck, the architectural paradigm of Processing-in-Memory (PIM), also referred to interchangeably as Compute-in-Memory (CIM), has transitioned from theoretical exploration to commercial silicon realization[2]. PIM architectures aim to collapse the memory-compute dichotomy by integrating logic directly into the memory substrate, thereby enabling operations to be executed *in situ* or strictly adjacent to the data cells[1]. This paradigm minimizes data movement, leverages the massive internal bit-level parallelism of memory arrays, and drastically reduces energy consumption while unlocking extreme bandwidth[6].

Designing an execution interface to compel a Random-Access Memory (RAM) chip to execute neural operations requires a multilayered, cross-stack approach. The implementation must bridge novel physical memory substrates—ranging from static and dynamic RAM to emerging non-volatile phase-change and resistive materials—with analog and digital circuit modifications, sophisticated interconnect protocols such as Compute Express Link (CXL), and high-level software compiler frameworks like the Multi-Level Intermediate Representation (MLIR) and the Open Neural Network Exchange (ONNX) Runtime[11].

1.1. Architectural Classification: Processing Using Memory vs. Processing Near Memory

The PIM taxonomy is broadly bisected into two fundamental design philosophies based on spatial integration, operational principles, and the degree to which they alter the memory cell itself: Processing Using Memory (PUM) and Processing Near Memory (PNM)[15].

Architecture Type	Operational Principle	Primary Hardware Mechanism	Advantages	Limitations
Processing Using Memory (PUM)	Computations are performed inside the memory array itself by exploiting the physical or analog properties of the memory cells and bitlines.	Simultaneous activation of multiple wordlines; crossbar array current summation; bitwise charge sharing on bitlines.	Maximum internal bandwidth utilization; near-zero data movement overhead; extreme density[9].	Often requires destructive reads; precision is limited by analog noise and Analog-to-Digital Converter (ADC) area overhead; operations are highly rigid[18].
Processing Near Memory (PNM)	Dedicated arithmetic logic units (ALUs) or digital processing cores are instantiated adjacent to memory banks or on the logic die of 3D-stacked memory modules.	High-speed intra-chip routing between memory banks and local digital ALUs or SIMD cores.	High precision (supports standard floating-point formats); high programmability; easier integration with existing software abstractions[20].	Still incurs localized data movement between the bank and the near-bank core; area overhead for placing logic near dense memory arrays[9].

Implementing a neural execution interface requires selecting the appropriate substrate based on the target application's precision, latency, and power constraints. While PUM directly leverages physical properties—such as Kirchhoff's circuit laws in resistive crossbars or charge sharing in dynamic capacitors—PNM relies on injecting standard digital logic into the memory

module's periphery, as seen in commercial architectures like UPMEM or High Bandwidth Memory-PIM (HBM-PIM)[15].

2. Hardware Substrates and Execution Interfaces

The physical medium of the RAM dictates the fundamental design of the execution interface. The implementation strategies diverge significantly depending on whether the target is embedded SRAM for edge inference, commodity DRAM for datacenter scalability, or emerging non-volatile memories for extreme-density analog computation.

2.1. SRAM-Based Compute-in-Memory Interfaces

Static Random-Access Memory (SRAM) constitutes the standard for on-chip cache and edge artificial intelligence (AI) accelerators due to its high speed, high endurance, and absolute compatibility with standard CMOS logic fabrication processes. Interfacing with SRAM for neural operations typically involves modifying the standard 6T (six-transistor) bitcell. Advanced implementations employ 8T, 8+T, 10T, or dual-wordline variants to decouple the read and write paths, thereby protecting the stored neural network weights from read-disturb phenomena during intensive computation[5]. SRAM IMC architectures are fundamentally divided into analog and digital modalities, each presenting distinct interface challenges and scaling trajectories.

Characteristic	Analog SRAM IMC	Digital SRAM IMC
Computation Domain	Voltage, Current, or Charge domain	Boolean logic (Digital domain)
MAC Execution	Simultaneous wordline assertion; bitline discharge summation	Multi-cycle shift-and-add logic; AND/XOR boolean gates
Peripheral Overhead	High (Requires large ADCs/DACs)	Low (Uses standard digital adders/accumulators)
Precision Support	Typically low (1-bit to 4-bit)	High (INT8, FP16, FP32)
PVT Variation Sensitivity	High (Susceptible to temperature, voltage drops)	Low (Immune to analog noise)
Process Scaling	Scales poorly beyond 16nm due to analog constraints	Scales excellently (e.g., TSMC 3nm, 4nm, 5nm)

Analog SRAM IMC interfaces perform multiply-accumulate (MAC) operations by

simultaneously asserting multiple wordlines. The bitcells discharge the vertical bitlines proportional to the data stored within them and the input activations, which are typically encoded as pulse-width or amplitude-modulated signals[5]. The aggregate current or voltage remaining on the bitline represents the analog partial sum of the MAC operation. While this approach massively amortizes the power consumption of memory reads across the array, it introduces severe interface bottlenecks at the array periphery. The analog partial sums must be converted back to the digital domain using Analog-to-Digital Converters (ADCs). These ADCs often consume up to 70% of the macro's total silicon area and power, rapidly eroding the efficiency gains achieved by the analog computation[3]. Furthermore, analog operations are highly susceptible to process, voltage, and temperature (PVT) variations, which degrade the signal margin and limit the achievable precision of the neural network[3].

To circumvent the ADC bottleneck and ensure robust operation at advanced technology nodes, the industry is increasingly favoring digital SRAM IMC. This interface executes MAC operations entirely via boolean logic gates integrated into or strictly adjacent to the memory array[3]. This approach simplifies multiplication into multi-cycle shift-and-add operations, reusing logic cells and avoiding the need for high-resolution ADCs[19]. Digital IMC maintains immunity to PVT variations, scales predictably with advanced nodes, and can support the high-precision floating-point formats (e.g., FP16, BF16) required for modern LLM inference and training[26].

Advanced digital interfaces implement internal write-back mechanisms, storing intermediate MAC results locally within the array to eliminate the latency and energy cost of routing partial sums back to a central accumulator[19]. Recent fabrications, such as TSMC's 3nm and 5nm digital CIM compilers, demonstrate the viability of this approach, yielding highly configurable macros that support varying bit-widths and parallel MAC architectures while achieving extreme energy efficiencies exceeding 120 TOPS/W[28].

2.2. Emerging Non-Volatile Memory: ReRAM and PCM Crossbars

Resistive Random-Access Memory (ReRAM) and Phase-Change Memory (PCM) represent the vanguard of analog PUM for dense neural acceleration[11]. Unlike charge-based volatile memories, these technologies store data as non-volatile resistance or conductance states, enabling instantaneous compute readiness without boot-time weight loading[11]. ReRAM relies on the formation and rupture of conductive filaments within a metal-insulator-metal stack, whereas PCM relies on the volumetric switching of chalcogenide materials (such as $Ge_2Sb_2Te_5$) between amorphous (high resistance) and crystalline (low resistance) phases[11].

In these architectures, the memory cells are arranged in a dense, two-dimensional crossbar array. The execution interface maps neural network weights to the precise conductance state (G) of the ReRAM or PCM cells[18]. Input activations from the preceding neural layer are converted to analog voltage pulses (V) by Digital-to-Analog Converters (DACs) and applied horizontally across the wordlines[22]. At each individual crosspoint, the current generated is

dictated strictly by Ohm's Law ($I_{ij} = V_i \times G_{ij}$). These individual currents flow vertically down the bitlines and sum naturally at the bottom of the array according to Kirchhoff's Current Law ($I_{j,out} = \sum_i I_{ij}$)[31].

This fundamental physical property allows a non-volatile crossbar to compute an entire Matrix-Vector Multiplication (MVM) in a single read cycle, effectively reducing the time complexity of the operation to $O(1)$ [22]. This provides throughputs and energy efficiencies orders of magnitude higher than digital counterparts[22]. State-of-the-art architectural frameworks utilizing this interface include PRIME (Processing in ReRAM-based Main Memory) and ISAAC (In-Situ Analog Arithmetic in Crossbars)[22]. PRIME is designed as a morphable architecture that configures portions of a ReRAM main memory chip as dedicated neural accelerators when needed, or as standard memory arrays otherwise, requiring minimal logic overhead[36]. Conversely, ISAAC utilizes dedicated ReRAM tiles connected via a Network-on-Chip (NoC) pipeline specifically optimized for Convolutional Neural Network (CNN) execution, aggressively pipelining data between crossbars to maximize computational density[22].

3. Subverting and Extending DRAM Execution Interfaces

Dynamic Random-Access Memory (DRAM) offers vastly superior capacity and density compared to SRAM, making it the primary target for accommodating the massive parameter counts of large-scale LLMs[38]. Interfacing with DRAM for neural execution involves either exploiting existing commodity DRAM through out-of-spec timing manipulations or designing specialized memory controllers for custom near-bank PIM DRAM.

3.1. Subverting Commodity DRAM: The PUM Approach

Neural operations can be executed on off-the-shelf, unmodified DRAM chips by replacing the standard host memory controller with a highly programmable, FPGA-based infrastructure, such as SoftMC (Soft Memory Controller) or DRAM Bender[39]. These testing platforms provide a low-level interface that allows developers to bypass standard JEDEC timing constraints and issue arbitrary DDR command sequences with fine-grained cycle control[41].

Through these interfaces, techniques like ComputeDRAM, Ambit, and RowClone demonstrate that memory arrays can perform logic[40]. The foundational mechanism relies on violating the standard row activation timing (t_{RAS}). By issuing multiple ACTIVATE (ACT) commands to different rows within the same DRAM bank in rapid succession, the memory controller forces multiple wordlines to open simultaneously[42]. The capacitors in these activated rows share their charge on the common bitline, which is initially precharged to exactly $V_{DD}/2$ [41].

Due to the inherent latching behavior of the DRAM sense amplifiers, this multi-row charge sharing inherently computes the majority function (MAJ) of the participating cells[42]. If a

programmer pre-sets one of the participating rows to a known state (zero or one), the MAJ function logically reduces to bitwise AND or OR operations[40]. These massively parallel, bit-serial primitives are then aggregated in software or via the memory controller's state machine to construct the complex arithmetic MAC operations required for neural network layers[40]. Frameworks like PiDRAM and EasyDRAM further wrap these low-level command sequences into higher-level C++ APIs, allowing system designers to evaluate end-to-end workload performance on unmodified DDR3 and DDR4 modules[6]. While highly innovative, this interface approach bypasses standard operating system virtual memory protections and cache coherency protocols, making it difficult to deploy in secure, multi-tenant cloud environments[40].

3.2. Commercial Near-Bank PIM Architectures

To provide a more standardized and deployable interface, memory vendors have developed specialized PNM architectures, such as Samsung's High Bandwidth Memory-PIM (HBM-PIM), SK Hynix's Accelerator-in-Memory (AiM), and the UPMEM architecture[13]. These architectures embed programmable computational units physically adjacent to the memory banks, exposing the immense internal bandwidth of the DRAM die to local processing elements[13].

The interface to these specialized chips is governed by specific register files mapped directly into the host's standard memory address space. In the Samsung HBM-PIM architecture, for instance, the interface relies on the following components:

- **Command Register File (CRF):** This register stores the specific PIM instructions (e.g., MAC, ADD, ReLU) that the near-bank SIMD units will execute[13].
- **General Register File (GRF):** This register acts as a local cache, holding input vectors and activation data to feed the ALUs continuously without repeatedly fetching from the dense DRAM arrays[13].
- **Scalar Register File (SRF) and Accumulate Register File (ARF):** These registers store scalar parameters and accumulate the partial sums generated during matrix multiplication[47].

To trigger a neural operation, the host memory controller writes a predefined instruction sequence into the CRF. When a specific memory address corresponding to the execution trigger is accessed via a standard DRAM READ or WRITE command, the PIM logic intercepts the command, hijacks the internal data bus, reads the row data into the GRF, executes the CRF instruction via the SIMD unit, and writes the output back to the ARF or the memory bank[13].

Similarly, SK Hynix's AiM architecture utilizes specialized MAC units and activation function hardware near GDDR6 banks[20]. To scale performance across multiple modules, the AiMX platform introduces an AiM Control Hub. This hub manages an instruction sequencer and a multicast interconnect, enabling the host to broadcast Complex Instruction Set Computer (CISC)-like instructions across all memory channels and banks simultaneously, amortizing command overhead and ensuring that the entire memory pool computes the neural layer in synchronized lockstep[48]. The UPMEM architecture takes a more generalized approach, embedding in-order DRAM Processing Units (DPUs) alongside a working RAM (WRAM) scratchpad and main RAM (MRAM), allowing the host CPU to offload distinct "tasklets" (software threads) compiled specifically for the DPU Instruction Set Architecture (ISA)[15].

4. Mitigating Analog Non-Idealities at the Interface

While analog crossbars theoretically provide massive efficiency gains, executing neural networks via analog physics introduces severe signal degradation and non-idealities that digital systems do not face. A functional execution interface must deploy physical and algorithmic compensation mechanisms to prevent catastrophic accuracy collapse.

4.1. The ADC Bottleneck and Precision Scaling

In ReRAM and PCM arrays, the analog current sum generated at the bottom of the bitlines must be digitized for subsequent layer processing, inter-chip routing, or non-linear activation functions. High-resolution ADCs are notoriously power-hungry and consume vast silicon area, frequently negating the density advantages of the memory cells themselves[22].

To optimize this peripheral interface, the system must employ bit-slicing. Because it is physically impossible to reliably map a full 8-bit or 16-bit weight to the conductance state of a single analog cell, the weight is mathematically sliced across multiple cells. For instance, an 8-bit weight might be distributed across four distinct cells, each holding 2 bits of information[22]. The input activations are similarly sliced in time or voltage amplitude. The resulting partial analog currents are digitized by lower-resolution, highly efficient ADCs (e.g., 4-bit). The digital outputs are subsequently shifted and added in the digital domain to reconstruct the full-precision mathematical output[24].

4.2. Conductance Drift and IR Drop

The non-volatile memory cells themselves are highly imperfect storage mediums for continuous analog weights. In Phase-Change Memory, the amorphous material undergoes continuous structural relaxation over time, causing the programmed conductance to drift lower[32]. This drift is modeled mathematically by a power-law relationship:

$G(t) = G_0(t/t_0)^{-\nu}$ where G_0 is the initial conductance programmed at time t_0, and ν is a cell-specific drift exponent that varies stochastically across the array[31]. If this phenomenon is unaccounted for, the neural network's inference accuracy will gracefully degrade over days and weeks as the weights systematically shift away from their trained values[53].

Furthermore, the metallic wires forming the rows and columns of the crossbar exhibit inherent parasitic resistance. As current travels down the bitline and across the wordline, a voltage drop—known as the IR drop—occurs. Consequently, memory cells located farther from the voltage source experience a lower effective voltage than those near the edge, skewing the Ohm's Law multiplication result and introducing spatial errors into the MVM computation[55].

4.3. Algorithmic Interfaces: Hardware-Aware and Quantization-Aware Training

To shield the neural network from these profound hardware variations, the software interface must integrate Hardware-Aware Training (HAT) and Quantization-Aware Training (QAT) long before the model is deployed to the chip[55].

Rather than training a model in standard 32-bit floating-point precision on a GPU and naïvely mapping the resulting weights to the analog crossbar, the training framework injects synthetic

noise simulating the physical non-idealities into the forward pass during the backpropagation training phase[60].

The algorithmic interface typically involves several systematic interventions:

1. **Noise Injection:** Thermal noise, shot noise, ADC quantization limits, read/write stochasticity, and array-specific IR drop profiles are simulated in software and added to the activations[30].
2. **QAT Formulation:** The weights and activations are constrained to discrete levels (e.g., INT4 or INT8) to match the limited resolution of the DACs and ADCs. The network learns to adjust its weights such that the classification loss function is minimized despite the aggressive quantization[60].
3. **Drift Compensation:** Advanced regularization techniques are applied during fine-tuning to penalize weight distributions that are highly susceptible to conductance drift. By incorporating the retention-induced weight variance as a loss penalty, the network is forced to rely on more stable, robust weight patterns[31].

Through these algorithmic interfaces, highly complex models, including ResNet architectures and Vision Transformers, can achieve near-iso-accuracy—meaning parity with their ideal FP32 digital baselines—even when deployed on physically noisy, drifting analog substrates[50].

5. System-Level Interconnects: The Compute Express Link (CXL)

Once the physical memory substrate is configured and the neural network is prepared, the PIM module must be integrated into the broader computing system. Historically, memory modules were strictly passive slaves to the CPU, interacting via rigid, synchronous JEDEC DDR standards[42]. Forcing a RAM chip to execute asynchronous neural code requires subverting or fundamentally expanding these interconnect protocols. For commercial, deployable PIM systems, the industry is rapidly coalescing around Compute Express Link (CXL) as the primary execution interface[12].

CXL operates over the physical and electrical layers of PCIe (PCI Express) 5.0 and 6.0, leveraging their immense bandwidth, but introduces specialized transaction layers that circumvent the severe memory-handling limitations of standard PCIe[67]. Under standard PCIe, memory semantics are highly inefficient; host CPUs cannot easily cache device memory, and DMA transfers require high software overhead and driver involvement, bottlenecking performance[68]. CXL resolves this by multiplexing three distinct sub-protocols over the link: CXL.io (for discovery, configuration, and legacy PCIe features), CXL.cache (allowing accelerators to cache host memory), and, crucially, CXL.mem[67].

5.1. The CXL.mem Protocol

The CXL.mem protocol exposes the internal RAM of the PIM device as standard, byte-addressable, cache-coherent memory to the host CPU[65]. A Type-3 CXL device utilizes CXL.mem to accept standard load/store instructions directly from the CPU, bypassing complex DMA setups[67].

To execute a neural operation via a CXL-attached PIM module, the CPU maps the PIM

instruction registers (such as the CRF in HBM-PIM) into its standard virtual address space. The CPU then writes a vector math instruction to that mapped address. The CXL interconnect carries this as a coherent memory write via CXL.mem flow control units (flits) directly to the PIM controller. The PIM controller intercepts the write, translates it to an internal analog or digital command, and triggers the near-bank ALUs[71]. This semantic interface allows unmodified host software to command specialized memory computations effortlessly.

5.2. Full-Duplex Exploitation and Scalable Fabrics

A profound architectural advantage of the CXL interface over traditional DDR memory buses is its physical full-duplex nature[73]. Standard DDR interfaces (including DDR4 and DDR5) operate on a half-duplex, shared-bus architecture. Because the command and data buses are shared, transitioning between reading and writing incurs heavy turnaround penalties, severely degrading bandwidth utilization when workloads mix read and write operations[74]. CXL, riding on the PCIe physical layer, utilizes physically separate transmit (TX) and receive (RX) lanes[73]. Advanced software interfaces, such as the CXLAimPod framework, actively monitor neural network dataflows and intelligently co-schedule memory requests to exploit this full-duplex capability[73]. By utilizing Extended Berkeley Packet Filter (eBPF) hooks in the Linux kernel, the framework can intercept memory operations and ensure that the CPU streams new activation data into the PIM device on the TX lanes simultaneously while the PIM device streams back the computed MAC results on the RX lanes[73]. This interface strategy unlocks up to a 61% bandwidth improvement over traditional memory buses at balanced read-write ratios[73]. Furthermore, the CXL 3.0 specification introduces memory pooling and port-based routing, enabling multiple distinct host CPUs to interface with a single massive PIM module via a switched fabric, vastly improving the scalability and resource utilization of distributed LLM inference[12].

6. Software Abstractions and Compilation Frameworks

Hardware integration and advanced interconnects are unusable without an accessible software interface. Machine learning engineers interact with high-level Python frameworks like PyTorch or TensorFlow, not CXL.mem flits or DDR timing diagrams[14]. A sophisticated compilation pipeline is required to lower high-level tensor operations into the bank-specific memory commands required by the RAM chip.

6.1. Multi-Level Intermediate Representation (MLIR) Dialects

Modern compiler infrastructures, particularly LLVM, are designed for traditional CPU/GPU architectures and struggle to represent the highly spatial, memory-centric, and distributed nature of PIM systems[77]. The Multi-Level Intermediate Representation (MLIR) framework addresses this deficiency by allowing compiler engineers to define custom, domain-specific "dialects"[14].

To target a PIM chip, a specialized MLIR dialect is constructed (for example, the pud dialect designed to represent bulk-wise in-DRAM compute primitives)[79]. The compilation interface operates progressively:

1. The high-level Python computation graph (e.g., a PyTorch torch.nn.Linear layer) is

imported into a high-level MLIR dialect, such as torch-mlir or linalg[80].

2. The compiler performs target-agnostic mathematical optimizations, such as loop unrolling, operation fusion, and symbolic dataflow recovery[81].
3. The intermediate representation is progressively lowered into the custom PIM dialect. Here, operations are split and mapped specifically to the physical constraints of the memory arrays[77].
4. For near-bank PIM devices, the compiler must dictate the exact physical data layout. It ensures that consecutive elements of a matrix are placed strictly within the same DRAM bank, because PIM ALUs generally cannot access data across different banks without routing it back through the host[83].
5. Finally, the backend translates the dialect into binary PIM instructions (targeting the CRF) and host-side memory instructions (targeting CXL.mem), yielding an executable sequence that optimally orchestrates the memory chip[13].

6.2. Runtime Framework Integration: ONNX Runtime and PyTorch

For dynamic runtime execution, the PIM module must be surfaced as a custom backend within standard machine learning frameworks.

In the ONNX (Open Neural Network Exchange) Runtime environment, the PIM interface is implemented as a custom C++ Execution Provider (EP) plugin[85]. When an ONNX graph is loaded, the framework partitions the workload. Standard operations, such as dynamic control flow or unsupported non-linearities, remain assigned to the default CPUExecutionProvider. Conversely, compute-heavy, memory-bound nodes like GEMV (General Matrix-Vector multiplication) are intercepted and offloaded to the PIMExecutionProvider[48]. The EP acts as the ultimate bridge, translating the ONNX nodes into the specific system-level API calls required to trigger the CXL or DDR memory controller[48].

Beyond standard ONNX integrations, advanced data-centric frameworks like PIMFlow (optimized for Convolutional Neural Networks) and PyGim (optimized for Graph Neural Networks) act as comprehensive extensions to PyTorch[76]. PIMFlow performs automated tensor placement and mixed-parallel scheduling, generating optimized command sequences that allow computation to overlap seamlessly between the host GPU and the PIM memory, maximizing system utilization[76]. In Graph Neural Network execution, PyGim intelligently classifies mathematical kernels based on their arithmetic intensity[89]. Compute-intensive kernels (e.g., neural layer updates) are routed to the host processor, while highly sparse, memory-bound kernels (e.g., edge feature aggregation) are pushed down to the distributed PIM nodes, leveraging the memory's internal bandwidth to eliminate the traditional bottlenecks associated with irregular graph traversals[89].

7. Security, Isolation, and Multi-Tenant Execution

As PIM modules transition from experimental academic prototypes to foundational datacenter infrastructure, managing multi-tenant security becomes a critical dimension of the execution interface. When multiple virtual machines (VMs) share a massive CXL 3.0 PIM pool, the memory controller must enforce strict isolation to prevent cross-process side-channel attacks and

unauthorized data access[46].
Traditional confidential computing mechanisms, such as Intel SGX or AMD SEV, encrypt data while it resides in RAM and decrypt it only inside the host CPU cache. If data computation is pushed *inside* the RAM, this standard memory encryption renders the data incomprehensible to the PIM ALUs, breaking the computational paradigm[92].
Interfaces like SecPIMlet are designed to provide trusted, multi-domain environments that reconcile confidential computing with in-memory execution[46]. This interface requires profound architectural shifts:

1. **PIM-Side Address Translation:** The memory management unit (MMU) logic must be partially migrated to the PIM device so that physical-to-virtual address translation boundaries are strictly enforced inside the memory chip itself, rather than relying solely on the host CPU[46].
2. **Context Switching Offload:** Secure thread execution contexts must be retained within the DRAM banks rather than saved back to the host CPU during a context switch, effectively eliminating data leakage across the vulnerable CXL bus[46].
3. **Spatio-Temporal Scheduling:** The PIM controller dynamically partitions memory banks. Specific banks are locked spatially to one tenant to prevent data contamination, while processing elements are time-sliced (temporal sharing) across domains, governed strictly by hardware-enforced access controls[46].

8. Conclusion

Establishing an interface to compel a RAM chip to autonomously execute neural network operations demands a profound, top-to-bottom transformation of the computing stack. At the physical layer, the selection between digital SRAM macros, near-bank DRAM architectures, or analog ReRAM and PCM crossbars dictates the fundamental limits of the system's precision, density, and latency[26]. To combat the inherent unreliability of analog substrates—such as ADC bottlenecks, spatial IR drops, and temporal conductance drift—sophisticated algorithmic interfaces, specifically hardware-aware and quantization-aware training, are mandated to preserve inference accuracy prior to deployment[54].
At the system level, the severe limitations of legacy half-duplex DDR protocols are being circumvented through software-defined memory controllers and the rapid adoption of the Compute Express Link (CXL) protocol. The CXL.mem interface effectively eliminates the cache-coherency barriers that historically impeded off-chip accelerators, providing a highly scalable, full-duplex conduit for seamless data movement[41]. Finally, this physical capability is rendered usable through advanced compiler infrastructures based on MLIR and runtime abstractions via ONNX Execution Providers and customized PyTorch frameworks, which intelligently partition workloads and abstract the intricate hardware topology away from the deep learning engineer[48]. Through this holistic architectural redesign, the immense potential of Processing-in-Memory can be fully realized, charting a definitive path to overcoming the von Neumann bottleneck.

Works cited

1. Performance Analysis of Digital Processing-in-Memory through a, https://arxiv.org/html/2305.04122v2
2. A Comprehensive Review of Processing-in-Memory Architectures, https://www.mdpi.com/2073-431X/13/7/174
3. In-memory computing accelerator for real-time image processing on, https://www.spiedigitallibrary.org/conference-proceedings-of-spie/13526/1352604/In-memory-computing-accelerator-for-real-time-image-processing-on/10.1117/12.3056338.full
4. A Modern Primer on Processing-In-Memory - arXiv, https://arxiv.org/html/2012.03112v5
5. In-Memory Computing Paradigm Using Energy-Efficient Analog, https://www.computer.org/csdl/journal/ec/2026/02/11435274/2eUghB2yoc8
6. PiDRAM: An FPGA-based Framework for End-to-end Evaluation of, https://arxiv.org/html/2206.00263v1
7. Daily Papers - Hugging Face, https://huggingface.co/papers?q=CPU%20inference
8. Accelerating Neural Network Inference with Processing-in-DRAM, https://arxiv.org/abs/2209.08938
9. Memory-Centric Computing: Recent Advances in Processing-in-DRAM, https://arxiv.org/html/2412.19275v1
10. A Fast Overlap-driven Mapping Framework for Processing In ... - arXiv, https://arxiv.org/html/2407.00604v1
11. Resistive Switching Random-Access Memory (RRAM): Applications, https://pubs.acs.org/chreay/article/125/12/5584/3691340/Resistive-Switching-Random-Access-Memory-RRAM
12. Opportunities and Challenges of CXL Memory Disaggregation, https://www.sciopen.com/article/10.26599/TST.2025.9010010
13. Hardware-software Co-design for Vector Similarity Search on HBM, https://jsts.org/jsts/XmlViewer/f449809
14. (PDF) MLIR: Scaling Compiler Infrastructure for Domain Specific, https://www.researchgate.net/publication/349993972_MLIR_Scaling_Compiler_Infrastructure_for_Domain_Specific_Computation
15. Analysis of Real Processing-in-Memory Hardware - arXiv, https://arxiv.org/pdf/2110.01709
16. Accelerating Neural Network Inference with Processing-in-DRAM, https://arxiv.org/pdf/2209.08938
17. Processing-in-Memory based CNN Acceleration - Chalmers ODR, https://odr.chalmers.se/bitstreams/da593f4a-681f-4b04-a0fc-b8d7d8ac3890/download
18. Unlocking Wordline-level Parallelism for Fast Inference on RRAM, https://taejunham.github.io/data/rram_iccad20.pdf
19. A Fully Digital SRAM-Based Four-Layer In-Memory Computing Unit, https://www.researchgate.net/publication/370310248_A_Fully_Digital_SRAM-Based_Four-Layer_In-Memory_Computing_Unit_Achieving_Multiplication_Operations_

and_Results_Store
20. An Experimental Evaluation of Machine Learning Training on a Real, https://arxiv.org/pdf/2207.07886
21. Accelerating Database Analytics with Bank-Level DRAM-PIM Filtering, https://arxiv.org/html/2504.06473v1
22. A Full System Stack Solution for Reconfigurable ReRAM-based NN, https://arxiv.org/pdf/1901.09904
23. A 65 nm 73 kb SRAM-Based Computing-In-Memory Macro With, https://www.researchgate.net/publication/359455213_A_65_nm_73_kb_SRAM-Based_Computing-In-Memory_Macro_With_Dynamic-Sparsity_Controlling
24. A 4-Kb 1-to-8-bit Configurable 6T SRAM-Based Computation-in, https://www.researchgate.net/publication/342940566_A_4-Kb_1-to-8-bit_Configurable_6T_SRAM-Based_Computation-in-Memory_Unit-Macro_for_CNN-Based_AI_Edge_Processors
25. In -Memory Processor With Unified FP/INT Pipeline for Cloud AI, https://www.researchgate.net/publication/365966002_ReDCIM_Reconfigurable_Digital_Computing-_In_-Memory_Processor_With_Unified_FPINT_Pipeline_for_Cloud_AI_Acceleration
26. A Multiply-Less Approximate SRAM Compute-In-Memory Macro for, https://www.researchgate.net/publication/383218231_A_Multiply-Less_Approximate_SRAM_Compute-In-Memory_Macro_for_Neural-Network_Inference
27. Hidehiro Fujiwara's research works | Taiwan Semiconductor, https://www.researchgate.net/scientific-contributions/Hidehiro-Fujiwara-2162686122
28. Yen-Huei Chen's research works | Taiwan Semiconductor, https://www.researchgate.net/scientific-contributions/Yen-Huei-Chen-2123381261
29. System-level Design and Modeling of an In ... - POLITesi - polimi, https://www.politesi.polimi.it/retrieve/dbd0b754-f214-4128-8360-4100c726086d/2026_03_Jreige_Thesis_01.pdf
30. Guidance and Control Neural Network Acceleration using Memristors, https://arxiv.org/html/2509.02369v1
31. Decoding Algorithms and HW Strategies to Mitigate Uncertainties in, https://repository.kaust.edu.sa/bitstreams/ab20e93a-77d1-425b-a9e3-cdb08db7659e/download
32. Phase-Change Memory for In-Memory Computing - ACS Publications, https://pubs.acs.org/chreay/article/125/11/5163/3691292/Phase-Change-Memory-for-In-Memory-Computing
33. Neural Network-Hardware Co-design for Scalable RRAM-based, https://arxiv.org/html/1811.02187v2
34. ISAAC: A Convolutional Neural Network Accelerator with In-Situ, https://users.cs.utah.edu/~rajeev/pubs/isca16-old.pdf
35. Efficient Processing-In-Memory with Neural Approximation of ... - arXiv, https://arxiv.org/pdf/2201.12861
36. PRIME: A Novel Processing-in-Memory Architecture for Neural,

https://www.computer.org/csdl/proceedings-article/isca/2016/8947a027/12OmNxwncHG
37. PRIME: A Novel Processing-in-Memory Architecture for Neural, https://www.researchgate.net/publication/309140450_PRIME_A_Novel_Processing-in-Memory_Architecture_for_Neural_Network_Computation_in_ReRAM-Based_Main_Memory
38. Evaluating Machine Learning Workloads on Memory-Centric, https://people.inf.ethz.ch/omutlu/pub/MLonUPMEM-PIM_ispass23.pdf
39. [PDF] PiDRAM: A Holistic End-to-end FPGA-based Framework for, https://www.semanticscholar.org/paper/PiDRAM%3A-A-Holistic-End-to-end-FPGA-based-Framework-Olgun-Luna/361b14bbd0345f404c3be486464fb3cb6c78aad4
40. PiDRAM: A Holistic End-to-end FPGA-based Framework for ... - arXiv, https://arxiv.org/html/2111.00082v6
41. DRAM Bender: An Extensible and Versatile FPGA-based ... - arXiv, https://arxiv.org/html/2211.05838v6
42. FracDRAM: Fractional Values in Off-the-Shelf DRAM, https://parallel.princeton.edu/papers/micro22-gao.pdf
43. EasyDRAM: An FPGA-based Infrastructure for Fast and Accurate, https://arxiv.org/html/2506.10441v2
44. ComputeDRAM In-Memory Compute using offshelf DRAM - Scribd, https://www.scribd.com/document/1057345352/ComputeDRAM-In-Memory-Compute-using-offshelf-DRAM
45. QUAC-TRNG: High-Throughput True Random Number Generation, https://www.mineshp.com/files/olgun_isca21_quac_paper.pdf
46. SecPIMlet: Confidential Computing for Spatio-temporally ... - uu .diva, https://uu.diva-portal.org/smash/get/diva2:2100558/FULLTEXT01.pdf
47. LP-Spec: Leveraging LPDDR PIM for Efficient LLM Mobile ... - arXiv, https://arxiv.org/html/2508.07227v3
48. AiMX: PIM for Generative AI Inference | PDF - Scribd, https://www.scribd.com/document/728615276/AiMX-Platform-Whitepaper-230906
49. pimDB: From Main-Memory DBMS to Processing-In-Memory DBMS, https://dblab.reutlingen-university.de/paper/2023_DaMoN_pimDB.pdf
50. Demonstration of transformer-based ALBERT model on a 14nm, https://pmc.ncbi.nlm.nih.gov/articles/PMC12485056/
51. An embedded PCM Peripheral Unit adding Analog MAC In-Memory, https://www.researchgate.net/publication/364692918_An_embedded_PCM_Peripheral_Unit_adding_Analog_MAC_In-Memory_Computing_Feature_addressing_Non-linearity_and_Time_Drift_Compensation
52. The Impact of Resistance Drift of Phase Change Memory (PCM, https://www.researchgate.net/publication/334288023_The_Impact_of_Resistance_Drift_of_Phase_Change_Memory_PCM_Synaptic_Devices_on_Artificial_Neural_Network_Performance
53. Aalborg Universitet Drift-Resilient PCM-Based Analog In-Memory,

https://vbn.aau.dk/files/835366099/Accepted_Author_Manuscript.pdf
54. ReRAM-aware Model Finetuning addressing I-V Non-linearity and, https://arxiv.org/html/2606.17471v3
55. Accelerating AI Applications using Analog In-Memory Computing, https://par.nsf.gov/servlets/purl/10262138
56. Energy-Efficient Neuromorphic Computing with CMOS-Integrated, https://escholarship.org/content/qt4jp0w0jc/qt4jp0w0jc_noSplash_165136ad10468ac3de069173fb2f2e4b.pdf?t=r6zeke
57. Modeling and Compensation of IR Drop in Crosspoint Accelerators, https://www.researchgate.net/publication/358185665_Modeling_and_Compensation_of_IR_Drop_in_Crosspoint_Accelerators_of_Neural_Networks
58. Reducing the Impact of Phase-Change Memory Conductance Drift, https://www.researchgate.net/publication/339266288_Reducing_the_Impact_of_Phase-Change_Memory_Conductance_Drift_on_the_Inference_of_large-scale_Hardware_Neural_Networks
59. A comprehensive review of advanced trends: from artificial synapses, https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2024.1279708/full
60. Statistical Computing Framework and Demonstration for In-memory, http://www.princeton.edu/~nverma/VermaLabSite/Publications/2022/ZhangDeavilleVerma_DAC2022.pdf
61. (PDF) Quantization-aware training for low precision photonic neural, https://www.researchgate.net/publication/363664899_Quantization-aware_training_for_low_precision_photonic_neural_networks
62. Reshape and Adapt for Output Quantization (RAOQ) - GitHub, https://raw.githubusercontent.com/mlresearch/v235/main/assets/zhang24i/zhang24i.pdf
63. Evaluation of Wafer-Scale SOT-MRAM for Analog Crossbar, https://www.studocu.vn/vn/document/truong-dai-hoc-bach-khoa-dai-hoc-quoc-gia-thanh-pho-ho-chi-minh/electrical-circuit/evaluation-of-wafer-scale-sot-mram-for-analog-crossbar-applications/163568860
64. Regularization-based Framework for Quantization-, Fault - arXiv, https://arxiv.org/html/2503.01297v1
65. CXL-Enabled Enhanced Memory Functions - IEEE Computer Society, https://www.computer.org/csdl/magazine/mi/2023/02/10015193/1JR6hjTTp04
66. Cxlalloc: Safe and Efficient Memory Allocation for a CXL Pod, https://www.cs.utexas.edu/~witchel/pubs/ni26asplos-cxlalloc.pdf
67. Opportunities and Challenges of CXL Memory Disaggregation, https://www.sciopen.com/local/article_pdf/10.26599/TST.2025.9010010.pdf
68. Extensive Exploration of Scalable Hardware Systems with CXL, https://www.usenix.org/system/files/fast26-an.pdf
69. A Novel Extensible Simulation Framework for CXL-Enabled Systems, https://arxiv.org/html/2411.08312v1
70. Persistent Memory vs RAM in 2025: CXL & NVDIMM-P Guide, https://corewavelabs.com/persistent-memory-vs-ram-cxl/

71. EP 4 671 968 A1 - European Publication Server web service, https://data.epo.org/publication-server/rest/v1.2/publication-dates/2025-12-31/patents/EP4671968NWA1/document.pdf
72. Toward Deployable CXL-PNM: The CMM-Ax Prototype and, https://www.computer.org/csdl/journal/tc/2026/04/11357537/2dkoUw99Ixa
73. CXLAimPod: CXL Memory is all you need in AI era - arXiv, https://arxiv.org/pdf/2508.15980
74. CXLAimPod: CXL Memory is all you need in AI era - ResearchGate, https://www.researchgate.net/publication/394920922_CXLAimPod_CXL_Memory_is_all_you_need_in_AI_era
75. The Next Five Years of Memory, And Why It Will Decide AI's Pace, https://medium.com/@Elongated_musk/the-next-five-years-of-memory-and-why-it-will-decide-ais-pace-27c4318fe963
76. PIMFlow: Compiler and Runtime Support for CNN Models on, https://yongwonshin.github.io/publications/pimflow.pdf
77. An MLIR-based Compilation Framework for CGRA Application, https://infoscience.epfl.ch/bitstreams/60b6cc4e-5ad5-4197-bf65-8af66d731894/download
78. DuoReduce: Bug Isolation for Multi-layer Extensible Compilation, https://web.cs.ucla.edu/~miryung/Publications/DuoReduce_FSE2025.pdf
79. Implementing an MLIR-based Compiler for In-DRAM Computing, https://cfaed.tu-dresden.de/files/Images/people/chair-cc/theses/2510_Zheng_MA.pdf
80. An Analog Compute-in-Memory Co-Processor for Low-Latency, https://openreview.net/pdf?id=BsGZ8p3VPp
81. CGO 2023: Proceedings of the 21st ACM/IEEE ... - SIGPLAN, https://www.sigplan.org/OpenTOC/cgo23.html
82. CINM (Cinnamon): A Compilation Infrastructure for ... - cfaed, https://cfaed.tu-dresden.de/files/Images/people/chair-cc/publications/2504_Khan_CINM_ASPLOS.pdf
83. A Tensor Compiler for Processing-In-Memory Architectures - arXiv, https://arxiv.org/html/2511.15503v1
84. DCC: Data-Centric Compilation of Machine Learning Kernels for, https://arxiv.org/html/2511.15503v5
85. Gibson, Perry (2023) Compiler-centric across-stack deep learning, https://theses.gla.ac.uk/83959/1/2023GibsonPhD.pdf
86. Extending the ONNX Runtime Framework for the Processing-in, https://www.researchgate.net/publication/359884564_Extending_the_ONNX_Runtime_Framework_for_the_Processing-in-Memory_Execution
87. Haris, Jude Christudas (2025) Hardware-software co-design of, https://theses.gla.ac.uk/85185/2/2025harisphd.pdf
88. Silent-PIM: Realizing the Processing-in-Memory Computing With, https://www.researchgate.net/publication/350000770_Silent-PIM_Realizing_the_Processing-in-Memory_Computing_With_Standard_Memory_Requests
89. PyGim : An Efficient Graph Neural Network Library for Real ... - arXiv,

https://arxiv.org/html/2402.16731v4
90. An Efficient Graph Neural Network Library for Real Processing-In, https://www.researchgate.net/publication/392522844_PyGim_An_Efficient_Graph_Neural_Network_Library_for_Real_Processing-In-Memory_Architectures
91. CV - Sunho Lee, https://myshlee417.github.io/cv/
92. UC Riverside - eScholarship.org, https://escholarship.org/content/qt3t82r6df/qt3t82r6df.pdf
93. PUBLICATIONS - CASYS Lab, https://casyslab.kaist.ac.kr/publications
94. UniSparse: An Intermediate Language for General Sparse Format, https://arxiv.org/pdf/2403.05802